Introduction to Internet of Things

Perception Layer - Part 2

RTOS fundamentals with ESP32

https://www.freertos.org

Content

1. Introduction
2. Justification
3. Tasks
4. Mutual exclusion
5. Queues
6. Practice

1. Introduction

What does “Real Time” mean in RTOS?

“Real-time" refers to the system’s ability to process data and respond to events within strict, predefined time constraints (deadlines). It emphasizes determinism (predictable timing) rather than raw speed, ensuring tasks complete within milliseconds or microseconds to prevent system failure. Key aspects of real-time in an RTOS include:

- Predictability (Deterministic Behavior): The OS guarantees that critical tasks will be completed within a specific timeframe, every time.
- Deadline Management: The correctness of the system depends not only on the logical result but also on the time the result is delivered. Missing a deadline is considered a failure.
- Hard vs. Soft Real-Time:
- Hard Real-Time: Missing a deadline results in catastrophic failure (e.g., airbags, flight control).
- Soft Real-Time: Missing a deadline reduces quality but is not catastrophic (e.g., video streaming, user interface updates).
- Event-Driven & Preemptive: The system can immediately prioritize critical tasks over less important ones, pausing lower-priority tasks (preemption) to handle urgent events.
- Low Jitter: RTOS ensures consistent, precise timing with minimal variation in task execution time.
- Real-time does not necessarily mean "instantaneous," but rather that the response occurs within the time necessary to control the environment.

1. Introduction

What is a Real Time Operating System?

A Real Time Operating System is an operating system that is optimised for use in embedded/real time applications. Their primary objective is to ensure a timely and deterministic response to events. An event can be external, like a limit switch being hit, or internal like a character being received.

Using a real time operating system allows a software application to be written as a set of independent tasks. Each task is assigned a priority and it is the responsibility of the Real Time Operating System to ensure that the task with the highest priority that is able to run is the task that is running. Examples of when a task may not be able to run include when a task is waiting for an external event to occur, or when a task is waiting for a fixed time period.

1. Introduction

What is a Real Time Kernel?

A Real Time Operating System can provide many resources to application writers - including TCP/IP stacks, files systems, etc. The Kernel is the part of the operating system that is responsible for task management, and intertask communication and synchronisation. FreeRTOS is a real time kernel.

What is a Real Time Scheduler?

Real Time Scheduler and Real Time Kernel are sometimes used interchangeably. Specifically, the Real Time Scheduler is the part of the RTOS kernel that is responsible for deciding which task should be executing.

2. Justification

Why use an RTOS?

You do not need to use an RTOS to write good embedded software. At some point though, as your application grows in size or complexity, the services of an RTOS might become beneficial for one or more of the reasons listed below. These are not absolutes, but opinion. As with everything else, selecting the right tools for the job in hand is an important first step in any project.

- Abstract out timing information
- Maintainability/Extensibility
- Modularity
- Cleaner interfaces
- Easier testing (in some cases)
- Code reuse
- Improved efficiency?
- Idle time
- Flexible interrupt handling
- Mixed processing requirements
- Easier control over peripherals

3 .Tasks

Tasks are implemented as C functions. Tasks must implement the expected function prototype shown in Listing 4.1. which takes a void pointer parameter and returns void.

```
void vATaskFunction( void * pvParameters );
```

3. Tasks

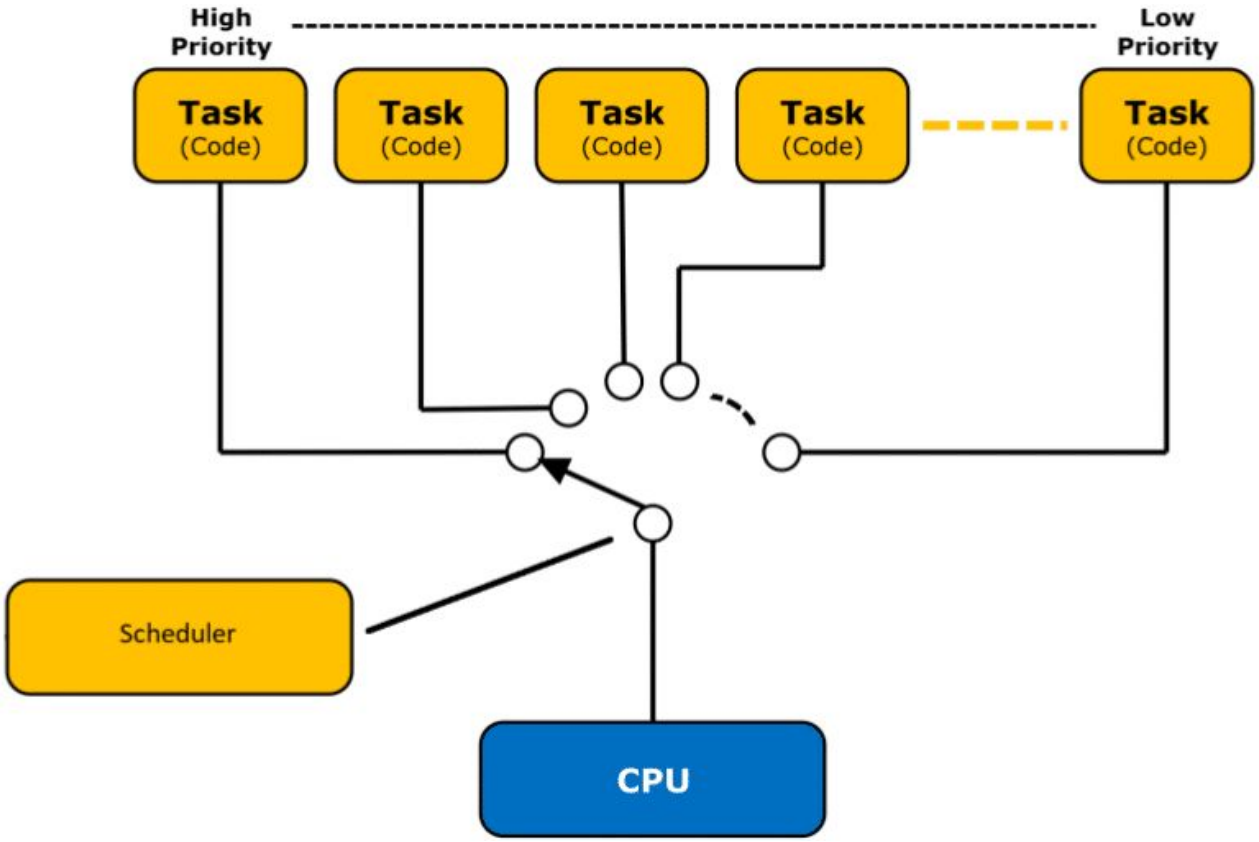
Tasks models process as an infinite loop

```
void vATaskFunction( void * pvParameters )
{
    /*
     * Stack-allocated variables can be declared normally when inside a function.
     * Each instance of a task created using this example function will have its
     * own separate instance of lStackVariable allocated on the task's stack.
     */
    long lStackVariable = 0;

    /*
     * In contrast to stack allocated variables, variables declared with the
`static`
     * keyword are allocated to a specific location in memory by the linker.
     * This means that all tasks calling vATaskFunction will share the same
     * instance of lStaticVariable.
     */
    static long lStaticVariable = 0;

    for( ;; )
    {
        /* The code to implement the task functionality will go here. */
    }

    /*
     * If the task implementation ever exits the above loop, then the task
     * must be deleted before reaching the end of its implementing function.
     * When NULL is passed as a parameter to the vTaskDelete() API function,
     * this indicates that the task to be deleted is the calling (this) task.
     */
    vTaskDelete( NULL );
}
```

3. Tasks

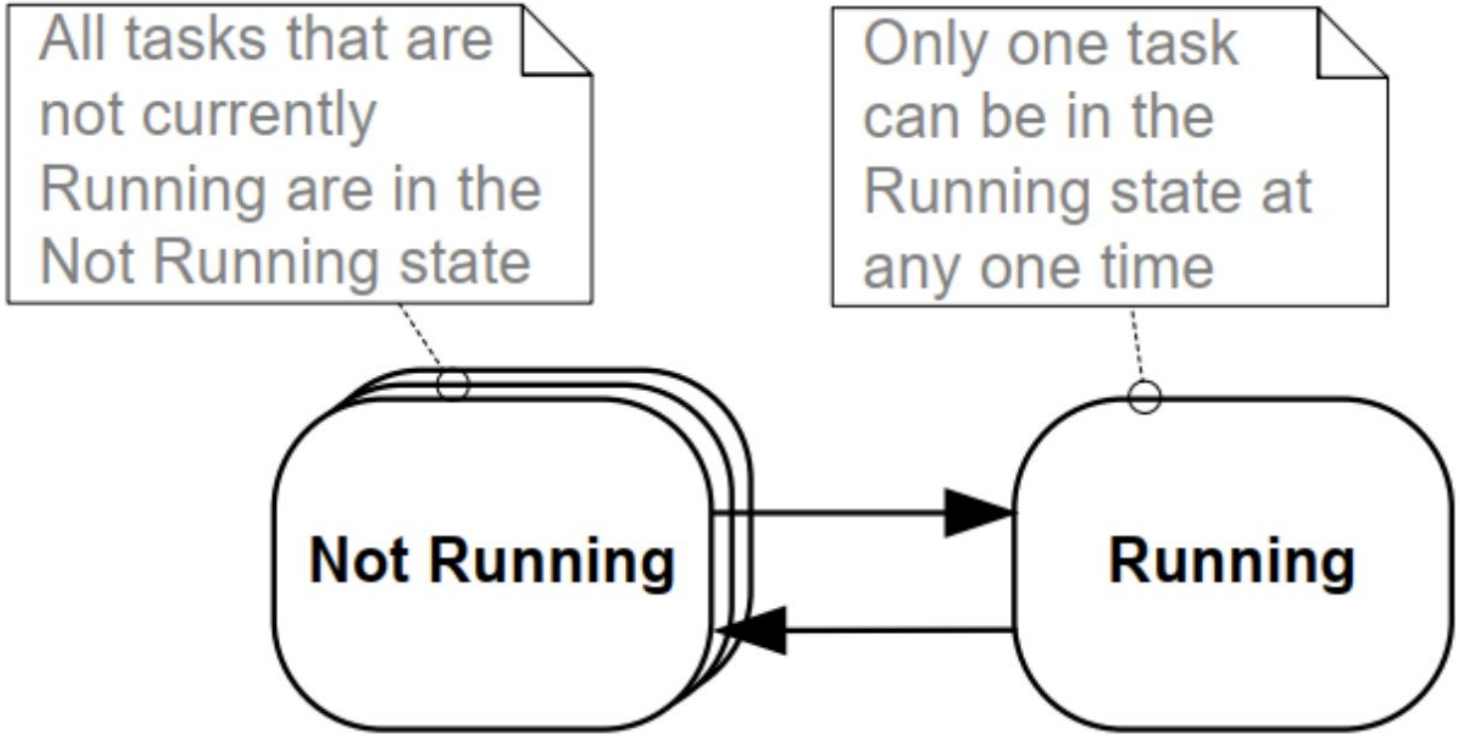
Scheduler

CPU as shared resource



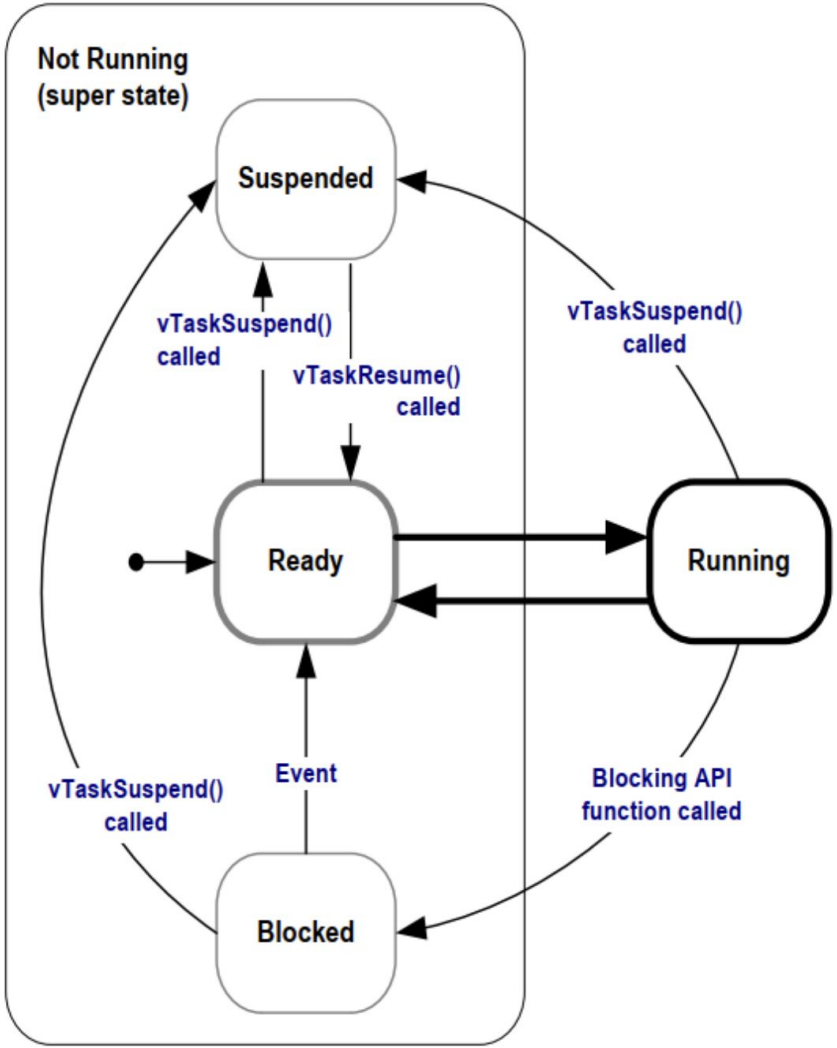
3 .Tasks

Task execution …



3. Tasks

Task execution … A more detailed scheme



3. Task

Task implementation

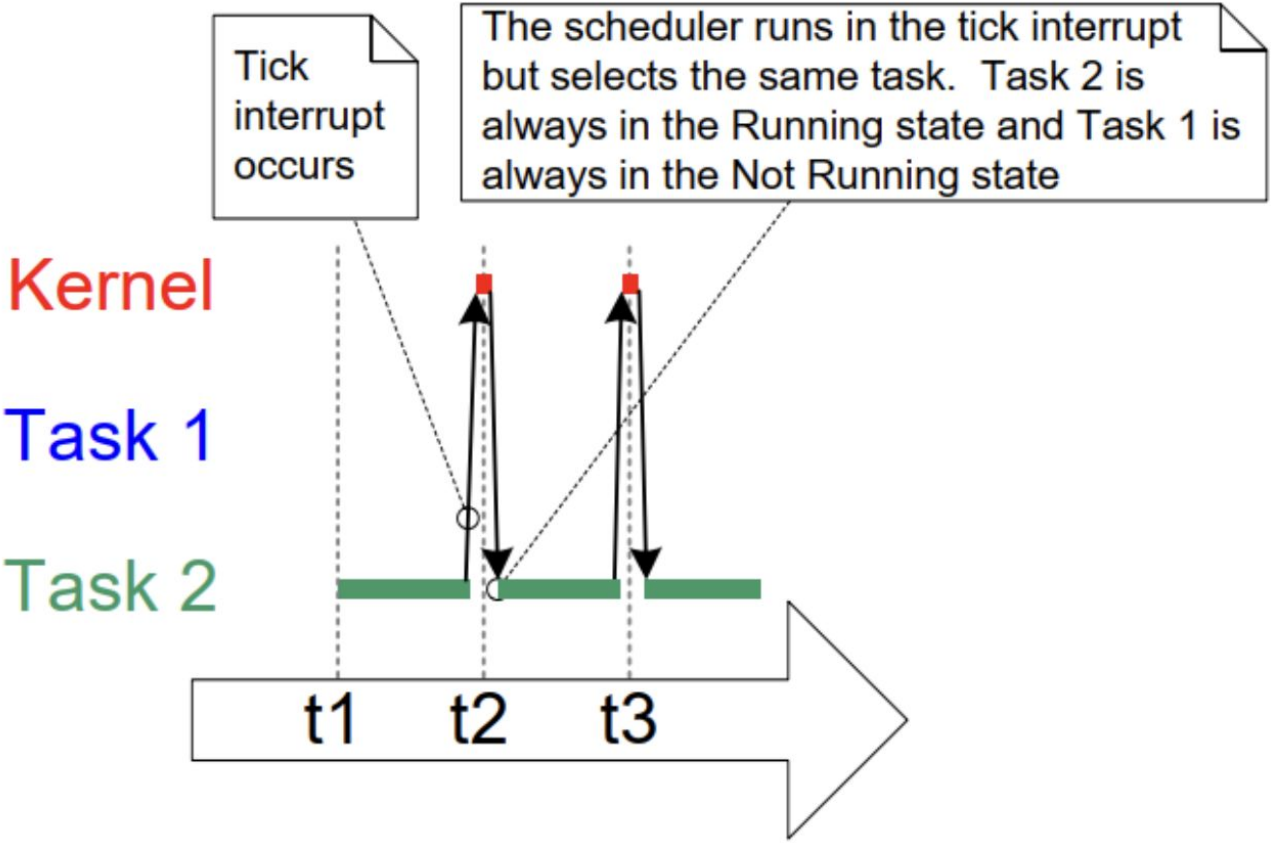
```c
/*
 * Define the strings that will be passed in as the task parameters.
 * These are defined const and not on the stack to ensure they remain valid
 * when the tasks are executing.
 */
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main( void )
{
    /* Create the first task with a priority of 1. */
    xTaskCreate( vTaskFunction,             /* Task Function    */
                 "Task 1",                  /* Task Name        */
                 1000,                      /* Task Stack Depth */
                 ( void * ) pcTextForTask1, /* Task Parameter   */
                 1,                         /* Task Priority    */
                 NULL );

    /* Create the second task at a higher priority of 2. */
    xTaskCreate( vTaskFunction,             /* Task Function    */
                 "Task 2",                  /* Task Name        */
                 1000,                      /* Task Stack Depth */
                 ( void * ) pcTextForTask2, /* Task Parameter   */
                 2,                         /* Task Priority    */
                 NULL );

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /* Will not reach here. */
    return 0;
}
```

4. Tasks

Pay attention to priorities

Task 1: Priority 1

Task 2: Priority 2



3. Tasks

Periodic task, with “CPU deliver”

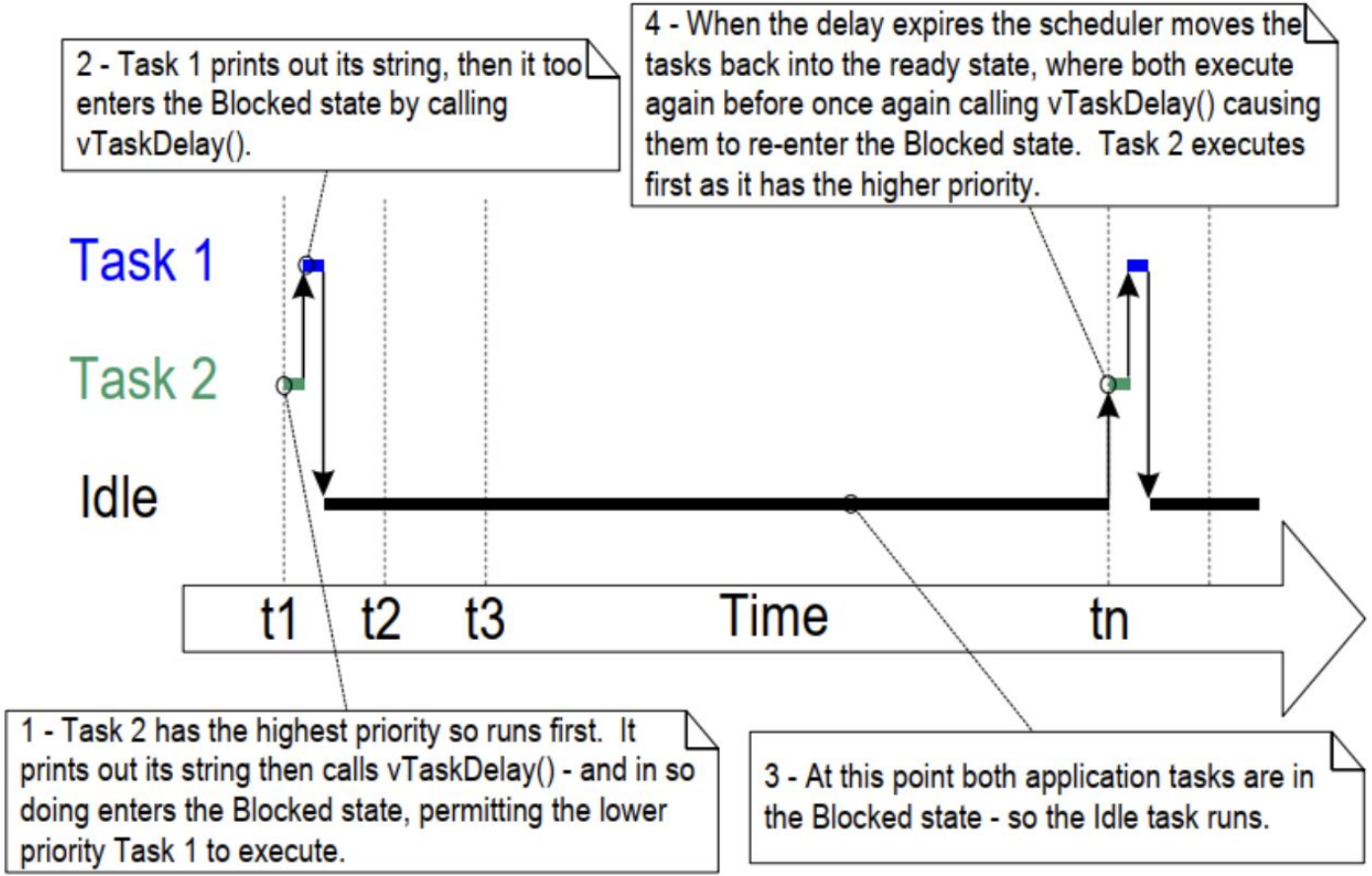
```c
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    /*
     * The string to print out is passed in via the parameter. Cast this to a
     * character pointer.
     */
    pcTaskName = ( char * ) pvParameters;

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintLine( pcTaskName );

        /*
         * Delay for a period. This time a call to vTaskDelay() is used which
         * places the task into the Blocked state until the delay period has
         * expired. The parameter takes a time specified in 'ticks', and the
         * pdMS_TO_TICKS() macro is used (where the xDelay250ms constant is
         * declared) to convert 250 milliseconds into an equivalent time in
         * ticks.
         */
        vTaskDelay( xDelay250ms );
    }
}
```

3. Tasks

Same function

2 tasks

Round robin



3. Tasks

Continuous processing tasks

```
void vContinuousProcessingTask( void * pvParameters )
{
    char * pcTaskName;

    /*
     * The string to print out is passed in via the parameter. Cast this to a
     * character pointer.
     */
    pcTaskName = ( char * ) pvParameters;

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /*
         * Print out the name of this task. This task just does this repeatedly
         * without ever blocking or delaying.
         */
        vPrintLine( pcTaskName );
    }
}
```

3. Tasks

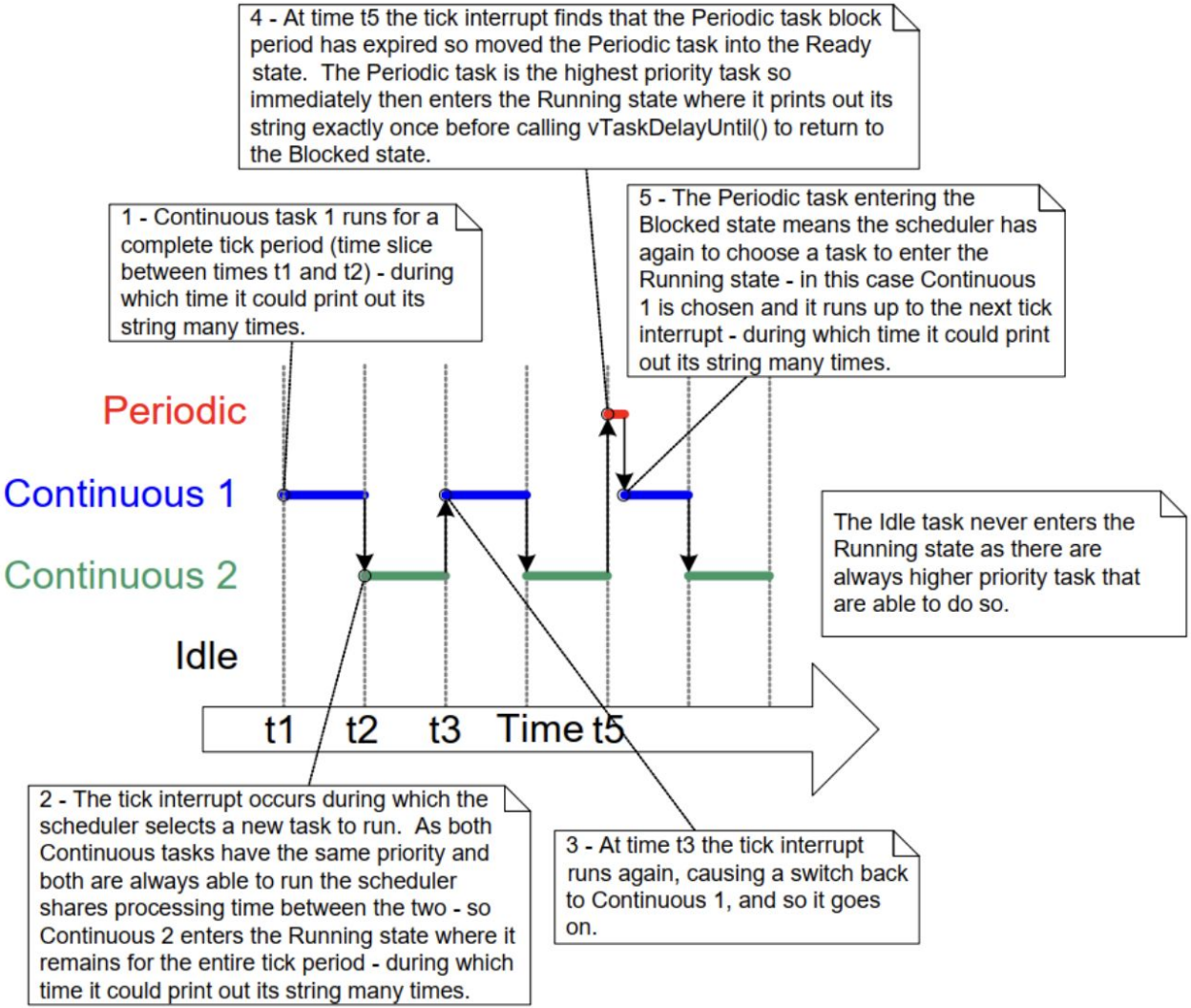
Periodic task

```c
void vPeriodicTask( void * pvParameters )
{
    TickType_t xLastWakeTime;

    const TickType_t xDelay3ms = pdMS_TO_TICKS( 3 );

    /*
     * The xLastWakeTime variable needs to be initialized with the current tick
     * count. Note that this is the only time the variable is explicitly
     * written to. After this xLastWakeTime is managed automatically by the
     * vTaskDelayUntil() API function.
     */
    xLastWakeTime = xTaskGetTickCount();

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintLine( "Periodic task is running" );

        /*
         * The task should execute every 3 milliseconds exactly - see the
         * declaration of xDelay3ms in this function.
         */
        vTaskDelayUntil( &xLastWakeTime, xDelay3ms );
    }
}
```

3. Tasks

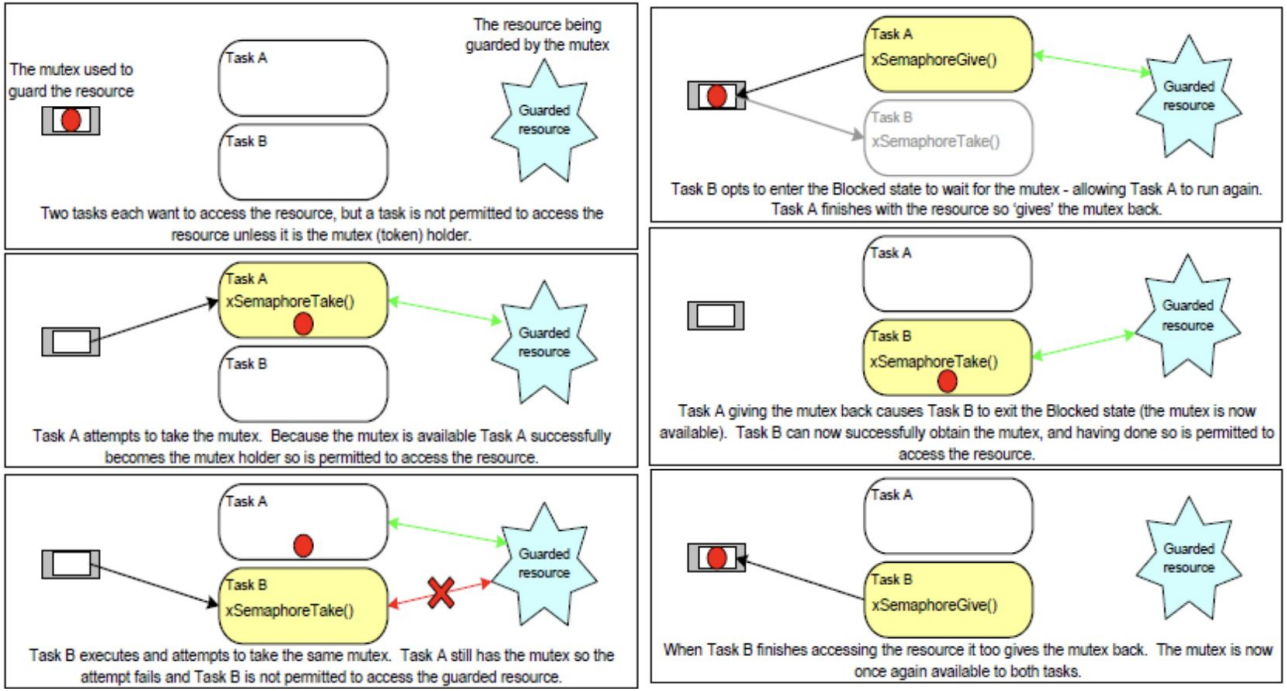
Continuous processing task and Periodic tasks execution

What about tasks priorities?



4. Mutual exclusion

For a task to access the resource legitimately, it must first successfully 'take' the token (be the token holder). When the token holder has finished with the resource, it must 'give' the token back. Only when the token has been returned can another task successfully take the token, and then safely access the same shared resource. A task is not permitted to access the shared resource unless it holds the token.



4. Mutual exclusion

No mutex behavior

```
NO_MUTEX: Task 2, count = 19
NO_MUTEX: Task 1, count = 20
NO_MUTEX: Task 1, count = 21
NO_MUTEX: Task 2, count = 21
NO_MUTEX: Task 1, count = 22
```

4. Mutex

This is a more complete example with:

- ESP32
- Platformio
- Espidf (no Arduino) style

Header part of main file

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_log.h"

#define TAG "MUTEX_EXAMPLE"

SemaphoreHandle_t xMutex;

int sharedCounter = 0;
```

4. Mutex

Create “competitive” tasks

```c
void task1(void *pvParameters)
{
    while (1)
    {
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            int local = sharedCounter;
            vTaskDelay(pdMS_TO_TICKS(100)); // Simula trabajo
            sharedCounter = local + 1;

            ESP_LOGI(TAG, "Task1 incrementó contador a %d", sharedCounter);

            xSemaphoreGive(xMutex);
        }

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

```c
void task2(void *pvParameters)
{
    while (1)
    {
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            int local = sharedCounter;
            vTaskDelay(pdMS_TO_TICKS(100)); // Simula trabajo
            sharedCounter = local + 1;

            ESP_LOGI(TAG, "Task2 incrementó contador a %d", sharedCounter);

            xSemaphoreGive(xMutex);
        }

        vTaskDelay(pdMS_TO_TICKS(700));
    }
}
```

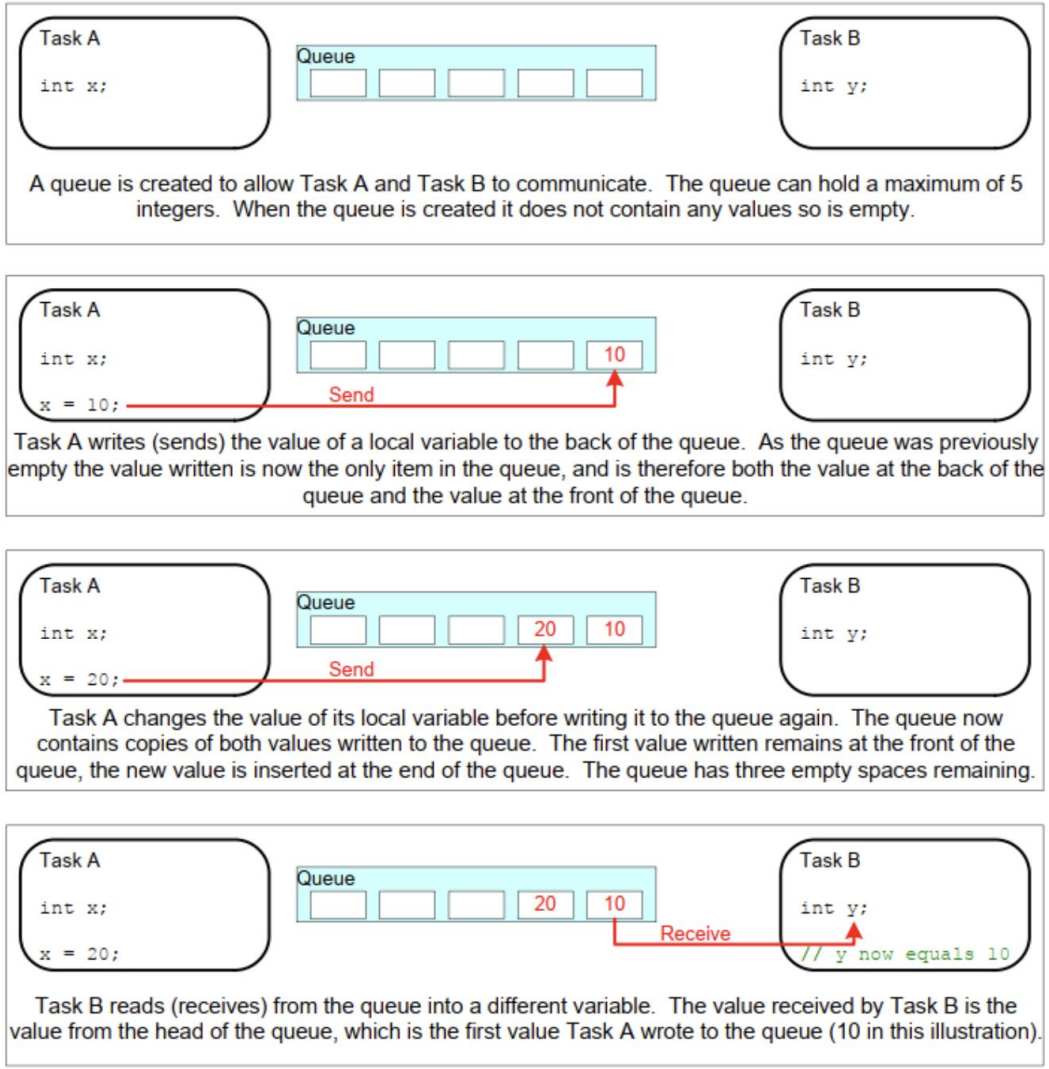
4. Mutex

main function

```
void app_main(void)
{
    xMutex = xSemaphoreCreateMutex();

    if (xMutex == NULL)
    {
        ESP_LOGE(TAG, "Error creando el mutex");
        return;
    }

    xTaskCreate(task1, "Task1", 2048, NULL, 2, NULL);
    xTaskCreate(task2, "Task2", 2048, NULL, 2, NULL);
}
```

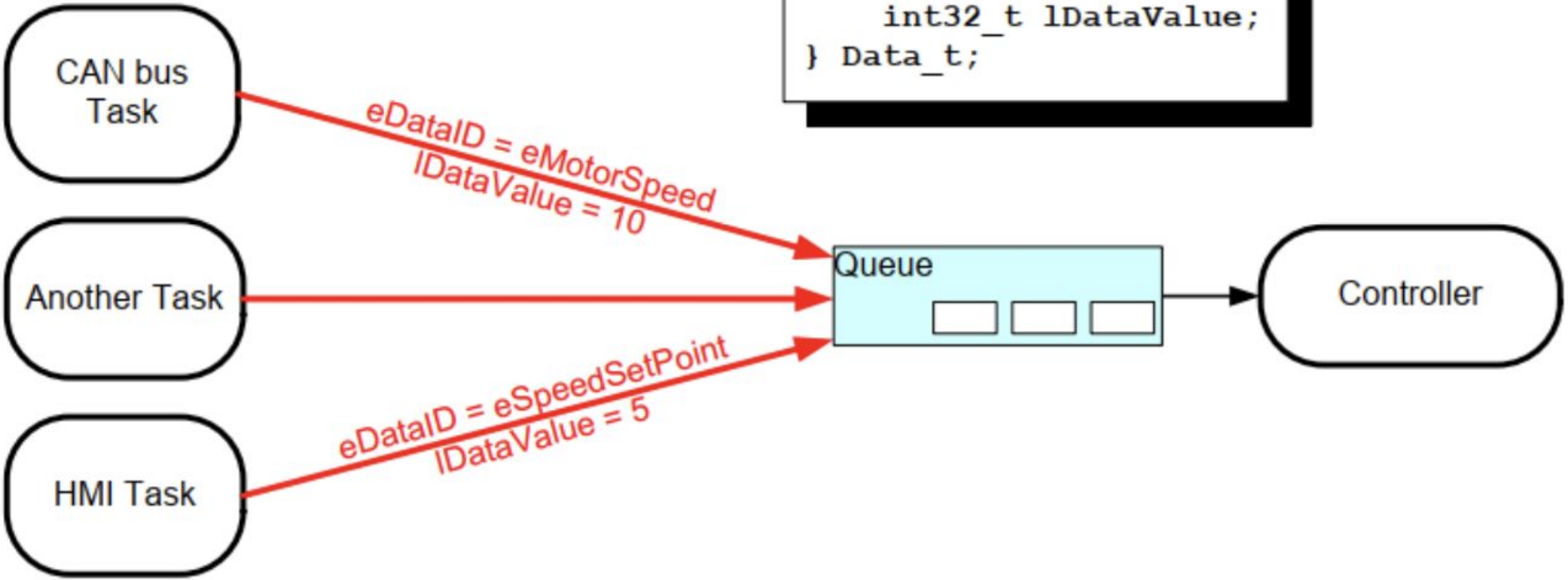
5. Queues

Queues are normally used as First In First Out (FIFO) buffers, where data is written to the end (tail) of the queue and removed from the front (head) of the queue. Figure 5.1 demonstrates data being written to and read from a queue that is being used as a FIFO. It is also possible to write to the front of a queue, and to overwrite data that is already at the front of a queue.



5. Queues

```
    int32_t lDataValue;
} Data_t;
```



5. Queues

This is a more complete example with:

- ESP32
- Platformio
- Espidf (no Arduino) style

Header part of main file

```c
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"

#define TAG "QUEUE_EXAMPLE"

#define QUEUE_LENGTH 10

QueueHandle_t dataQueue;
```

```c
typedef enum {
    DATA_INT,
    DATA_STRING
} data_type_t;

typedef struct {
    data_type_t type;
    union {
        int intValue;
        char strValue[32];
    } data;
} queue_message_t;
```

5. Queues

Create sender tasks

```c
void producerTask1(void *pvParameters)
{
    queue_message_t msg;
    int counter = 0;

    while (1)
    {
        msg.type = DATA_INT;
        msg.data.intValue = counter++;

        if (xQueueSend(dataQueue, &msg, portMAX_DELAY) == pdPASS)
        {
            ESP_LOGI(TAG, "Task1 envió entero: %d", msg.data.intValue);
        }

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```c
void producerTask2(void *pvParameters)
{
    queue_message_t msg;
    int counter = 0;

    while (1)
    {
        msg.type = DATA_STRING;
        snprintf(msg.data.strValue, sizeof(msg.data.strValue),
                 "Mensaje %d", counter++);

        if (xQueueSend(dataQueue, &msg, portMAX_DELAY) == pdPASS)
        {
            ESP_LOGI(TAG, "Task2 envió string: %s", msg.data.strValue);
        }

        vTaskDelay(pdMS_TO_TICKS(1500));
    }
}
```

5. Queues

Create receiver task

```
void consumerTask(void *pvParameters)
{
    queue_message_t receivedMsg;

    while (1)
    {
        if (xQueueReceive(dataQueue, &receivedMsg, portMAX_DELAY) == pdPASS)
        {
            switch (receivedMsg.type)
            {
                case DATA_INT:
                    ESP_LOGI(TAG, "Consumer recibió entero: %d",
                             receivedMsg.data.intValue);
                    break;

                case DATA_STRING:
                    ESP_LOGI(TAG, "Consumer recibió string: %s",
                             receivedMsg.data.strValue);
                    break;
            }
        }
    }
}
```

5. Queues

main function

```
void app_main(void)
{
    dataQueue = xQueueCreate(QUEUE_LENGTH, sizeof(queue_message_t));

    if (dataQueue == NULL)
    {
        ESP_LOGE(TAG, "Error creando la cola");
        return;
    }

    xTaskCreate(producerTask1, "Producer1", 2048, NULL, 2, NULL);
    xTaskCreate(producerTask2, "Producer2", 2048, NULL, 2, NULL);
    xTaskCreate(consumerTask,  "Consumer",  2048, NULL, 1, NULL);
}
```

6. Practice

- Install Platformio complement on VSCode
- Create and test Round-Robin application with 2 tasks
  - Printing messages
- Create and test Mutex application
  - Printing a counter
- Create a Queue application
  - Use the example